TalkHive

Online Chat System

Project Handoff Guide

Stack
React Native (Expo) + Supabase + TypeScript

Purpose
This document gives any new developer, team lead, or stakeholder everything they need to understand, run, and continue building TalkHive.

Version: 1.0.0 **Status:** In Development **Platform:** iOS & Android

1. Project Overview

TalkHive is a real-time mobile chat application built for iOS and Android using React Native Expo. It uses Supabase as the sole backend service — no custom server is needed. Users register with email and password, join public or private chat rooms, send direct messages, share media, and see who is online in real time.

App Name	TalkHive — Online Chat System
Platform	iOS and Android (Expo managed workflow)
Frontend	React Native + Expo + TypeScript
Backend	Supabase (Auth, Database, Realtime, Storage)
Navigation	Expo Router (file-based routing)
State	Zustand stores per feature slice
Auth	Email and password only (no OAuth)
Architecture	Vertical slice — each feature is self-contained

1.1 Core Features

- Email and password registration, login, and password reset
- Post-registration profile setup (username, display name, avatar)
- Public and private chat rooms — create, join, manage
- Real-time messages via Supabase Realtime WebSockets
- 1-on-1 direct messages with read receipts
- Online presence indicators and typing indicators
- Image and file sharing stored in Supabase Storage
- Emoji reactions on messages
- Full-text message search (PostgreSQL FTS)
- Expo push notifications for new messages and mentions

2. Prerequisites

Make sure the following are installed and configured before running the project.

Tool	Required Version	Notes
Node.js	18.x or higher	Use nvm for version management
npm or yarn	npm 9+ / yarn 1.22+	npm comes with Node
Expo CLI	Latest (npx expo)	No global install needed

EAS CLI	Latest	npm install -g eas-cli
Expo Go (device)	Latest from App Store	For development testing
Supabase account	Free tier or higher	supabase.com
Git	2.x or higher	Version control

3. Environment Setup

3.1 Clone the Repository

```
git clone https://github.com/your-org/talkhive.git
cd talkhive
npm install
```

3.2 Environment Variables

Create a .env file at the project root. Copy .env.example and fill in your Supabase credentials.

```
EXPO_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=your-anon-key-here
```

Where to find these: Go to Supabase Dashboard → your project → Settings → API. Copy the Project URL and anon/public key.

3.3 Supabase Database Setup

Run the schema SQL once to set up all tables, RLS policies, triggers, and storage buckets.

1. Open your Supabase project dashboard
2. Navigate to SQL Editor → New Query
3. Paste the entire contents of talkhive_schema.sql
4. Click Run — the schema is ready

Important: The schema creates the profiles table trigger automatically. Every new auth.users row immediately gets a matching profiles row — no manual step needed.

3.4 Supabase Storage

The schema SQL already creates two buckets:

- avatars — public bucket for profile pictures
- attachments — private bucket for chat media files

No extra configuration is needed in the Storage dashboard.

4. Running the App

4.1 Development

```
npx expo start
```

This starts the Metro bundler. From here you can:

- Press i to open the iOS simulator
- Press a to open the Android emulator
- Scan the QR code with Expo Go on a physical device

4.2 Run on Physical Device

Install Expo Go from the App Store or Google Play. Make sure your device and development machine are on the same Wi-Fi network, then scan the QR code shown in the terminal.

4.3 Production Build (EAS)

```
eas build --platform ios
eas build --platform android
eas build --platform all
```

EAS Build produces .ipa (iOS) and .apk/.aab (Android) files for store submission. Configure eas.json before your first build.

5. Folder Structure

TalkHive uses a vertical slice architecture. Every feature screen owns its UI components and data services. Shared code lives in global layers.

Folder	Purpose
src/screens/<Feature>/	Feature slice — screens, local components, services
src/components/	Shared UI components (Button, Input, Avatar, Toast...)
src/components/chat/	Chat-specific shared UI (MessageBubble, MessageInput...)
src/services/core/	Supabase client, realtime, storage, upload
src/services/notifications/	Expo push token and notification sending

src/services/presence/	Online presence tracking
src/state/	Zustand stores per feature (chatStore, dmStore...)
src/context/	React context providers (Auth, Theme, Toast, Presence)
src/hooks/	Shared custom hooks (useRealtimeMessages, useToast...)
src/navigation/	Stack and tab navigators
src/styles/	Global tokens — colors, fonts, spacing, shadows
src/utils/	Helpers, formatters, validation, date utils
src/config/	Supabase config, env, routes, feature flags
assets/	Fonts, images, icons, Lottie animations

5.1 Feature Slice Pattern

Every screen folder follows this consistent structure:

- screens/FeatureName/FeatureName.jsx — main screen component
- screens/FeatureName/FeatureName.styles.js — StyleSheet for the screen
- screens/FeatureName/components/ — UI components used only by this feature
- screens/FeatureName/services/ — Supabase queries and business logic for this feature

This means a developer can find everything about a feature in one place without hunting across the codebase.

6. Architecture

6.1 Data Flow

The app follows a unidirectional data flow pattern:

5. User action triggers a function in the screen component
6. Screen calls the feature's service (e.g. chatService.sendMessage)
7. Service calls Supabase (insert, select, update, delete)
8. Supabase Realtime pushes changes back to all subscribed clients
9. The Zustand store is updated with the new data
10. React re-renders the UI from the updated store

6.2 Realtime Subscriptions

Supabase Realtime is used for three live channels:

- messages channel — new, edited, and deleted messages in a room
- direct_messages channel — new DMs between two users
- profiles channel — online/offline presence changes

Subscriptions are created when a screen mounts and cleaned up when it unmounts. The hooks useRealtimeMessages, useRealtimePresence, and useRealtimeTyping handle subscription lifecycle.

6.3 Authentication Flow

Auth is handled by Supabase Auth with JWT sessions:

11. User registers with email, password, and username
12. Supabase creates the auth.users row and triggers profile creation
13. User is redirected to ProfileSetup to pick an avatar
14. On subsequent opens, AuthContext checks the active session
15. Expired sessions are auto-refreshed by the Supabase client
16. Logout calls supabase.auth.signOut() and clears all stores

6.4 Row Level Security

All tables have RLS enabled. Key rules:

- Users can only read messages from rooms they are members of
- Users can only read direct messages where they are the sender or recipient
- Users can only update or delete their own messages
- Private rooms are invisible to non-members
- Push tokens are private — users can only access their own

Dev tip: If a query returns no data unexpectedly during development, check RLS policies first. Disable RLS temporarily on a table in the Supabase dashboard to confirm it is a policy issue, then fix the policy.

7. Key Dependencies

Package	Version	Purpose
expo	~51.x	Core Expo SDK and build tooling
react-native	0.74.x	Mobile UI framework
expo-router	~3.x	File-based navigation

@supabase/supabase-js	^2.x	Supabase client — auth, DB, realtime, storage
zustand	^4.x	Lightweight global state management
expo-image-picker	~15.x	Camera roll and camera access
expo-notifications	~0.28.x	Push notification handling
expo-file-system	~17.x	File read/write for uploads
react-native-gifted-chat	^2.x	Chat UI components base
@shopify/flash-list	^1.x	Performant virtualized list
expo-secure-store	~13.x	Secure storage for tokens
dayjs	^1.x	Lightweight date formatting

8. Database Schema Summary

The full schema SQL is in talkhive_schema.sql. Below is a quick reference of all tables.

Table	Key Columns	Purpose
profiles	id, username, avatar_url, is_online	Public user info — extends auth.users
rooms	id, name, is_private, created_by	Chat channels (public or private)
room_members	room_id, user_id, role	Room membership and roles
messages	id, room_id, sender_id, body	All room chat messages
direct_messages	id, from_id, to_id, body, is_read	1-on-1 private messages
attachments	message_id, dm_id, storage_path	File references for messages and DMs
reactions	message_id, user_id, emoji	Emoji reactions on messages
push_tokens	user_id, token	Expo push tokens per device

9. Screens Reference

Screen	Route / File	Description
Register	Auth/Register.jsx	Email, password, username signup
Login	Auth/Login.jsx	Email and password login

Forgot Password	Auth/ForgotPassword.jsx	Send password reset email
Onboarding	Onboarding/WelcomeTour.jsx	First-time welcome slides
Profile Setup	ProfileSetup/ProfileSetup.jsx	Username and avatar after signup
Room List	Rooms/RoomList.jsx	Browse and join chat rooms
Chat Room	Chat/ChatRoom.jsx	Live group chat in a room
Room Info	Chat/RoomInfo.jsx	Room settings and member list
DM Inbox	DirectMessages/DMInbox.jsx	List of DM conversations
DM Conversation	DirectMessages/DMConversation.jsx	1-on-1 private chat
Profile	Profile/Profile.jsx	View and edit own profile
Search	Search/Search.jsx	Search messages and users
Notifications	Notifications/NotificationInbox.jsx	Notification history
Settings	Settings/Settings.jsx	App, notification, privacy settings

10. Development Conventions

10.1 File Naming

- Screen components — PascalCase.jsx (e.g. ChatRoom.jsx)
- Style files — PascalCase.styles.js paired with the component
- Hooks — camelCase starting with use (e.g. useRealtimeMessages.js)
- Services — camelCase ending with Service (e.g. chatService.js)
- Stores — camelCase ending with Store (e.g. chatStore.js)
- Utilities — camelCase descriptive names (e.g. formatters.js)

10.2 Coding Standards

- TypeScript strict mode — no implicit any
- Functional components only — no class components
- StyleSheet.create for all styles — no inline style objects
- Every Supabase call wrapped in try/catch with error logging
- Realtime subscriptions cleaned up in useEffect return function
- All user-facing strings extracted to a constants file (future i18n ready)

10.3 Branch Strategy

- main — production-ready code only
- develop — integration branch for completed features
- feature/feature-name — individual feature branches off develop

- fix/issue-description — bug fix branches

Pull requests require at least one reviewer approval before merging into develop.

11. Push Notifications

TalkHive uses Expo Push Notifications. The flow is:

17. On login, the app requests notification permission from the device
18. If granted, Expo generates a push token for that device
19. The token is saved to the push_tokens table in Supabase
20. A Supabase Edge Function listens for new messages via a database webhook
21. The Edge Function calls the Expo Push API with the recipient's token
22. The notification appears on the device even when the app is closed

iOS requirement: Push notifications on iOS require a paid Apple Developer account and a provisioning profile with push entitlements. Configure this in eas.json before building for iOS.

12. Known Limitations & Future Work

Current Limitation	Planned Fix / Future Feature
No voice or video calls	Integrate Daily.co or LiveKit in v2
No message threading	Add thread reply support in v1.1
File uploads limited to images	Extend to PDF and document uploads
No admin moderation panel	Build web-based admin dashboard
No message translation	Integrate translation API in v2
Single language (English)	Add i18n support with i18next

13. Troubleshooting

Messages not appearing in real time

Check that Supabase Realtime is enabled for the messages table. Go to Supabase Dashboard → Database → Replication and confirm the messages table is added to the supabase_realtime

publication. Also verify the Realtime subscription in chatRealtimeService.js is using the correct channel name and room_id filter.

RLS blocking queries during development

If a select query returns an empty array despite data existing in the table, RLS is likely blocking access. Temporarily disable RLS on the table in Supabase Dashboard → Authentication → Policies to confirm, then review the policy logic. Never leave RLS disabled in production.

Expo Go crashing on start

Run npx expo start --clear to clear the Metro cache. If the issue persists, delete node_modules and run npm install again. Ensure your Expo Go app version matches the Expo SDK version in package.json.

Push notifications not received on iOS

Confirm the Expo push token is being saved correctly to the push_tokens table. Check the Edge Function logs in Supabase Dashboard → Edge Functions for errors. On a physical device, ensure notifications are allowed in iOS Settings for the app.

Avatar not loading after upload

Verify the avatars storage bucket is set to public. Check that the avatar_url stored in the profiles table is the full public URL returned by supabase.storage.from('avatars').getPublicUrl(), not just the file path.

14. Contacts & Ownership

Role	Name	Responsibility
Project Owner	— fill in —	Product decisions, roadmap
Lead Developer	— fill in —	Architecture, code reviews, releases
Backend / Supabase	— fill in —	Schema changes, RLS policies, Edge Functions
UI / Frontend	— fill in —	Screens, components, styles
QA	— fill in —	Testing, bug triage

Last updated: Fill in the current date when handing off this document.